Resilient Reinforcement Learning for Traffic Signal Control

Detecting, Diagnosing and Mitigating Distribution Shift in Real-Time Urban Intersections

---

Document: Comprehensive Team Contributions, Architecture, and Verification Audit Report
Institution: Department of Artificial Intelligence, Amrita Vishwa Vidyapeetham
Faculty Guide: [GUIDE NAME]
Review Date: 03 November 2026, 10:00 AM
Status: Fully Verified Empirical Baseline & Closed-Loop Architecture

Part	Team Member	Roll Number	Assigned Subsystem & Focus	Equal Standing Note
Part 1	**Ayyappa Reddy**	AM.SC.U4AIE23046	SUMO TraCI Environment, DQN Training Loop, Classical Baselines, Max-Pressure Left-Turn Fix & Fairness Parameterization	Primary Presenter: Environment & Baselines
Part 2	**Sriram**	AM.SC.U4AIE23014	4-Way MDP Shift Injection Engine (S/T/A/R), 3-Sigma Telemetry Monitor, Random Forest Root-Cause Diagnostic Classifier	Primary Presenter: Shifts, Monitor & Diagnosis
Part 3	**Vivek Reddy**	AM.SC.U4AIE23050	Targeted Shift Mitigations (Observation, Transition, Action Safety-Gate), Fallback Governor, Closed-Loop Supervisor Pipeline	Primary Presenter: Mitigations & Controller

Governance & Integrity Statement:
All three team members have strictly equal academic standing in this project; no hierarchical titles apply. The structure Part 1 (Ayyappa Reddy), Part 2 (Sriram), and Part 3 (Vivek Reddy) reflects the natural chronological and physical data flow of the system: from the microscopic simulation and nominal policy, through runtime disturbance injection and monitoring, to closed-loop adaptive mitigation and fallback control. Every empirical figure in this report has been verified against raw artifacts on disk using Eclipse SUMO 1.27.1 and Python 3.11.

Table of Contents

Section 1: Project Overview in Plain Language	Page 3
Section 2: System Architecture & Component Interactions	Page 4
Section 3: Subsystem Integration & End-to-End Data Flow	Page 5
Section 4: Individual Member Contributions & Technical Deep Dives	Page 6
• **Part 1:** Ayyappa Reddy — SUMO, DQN Training, Baselines & Max-Pressure Fix	Page 6
• **Part 2:** Sriram — Shift Injection Engine, 3-Sigma Monitor & Diagnostic Classifier	Page 9
• **Part 3:** Vivek Reddy — Shift Mitigations, Fallback Governor & Closed-Loop Controller	Page 12
Section 5: Shared Team Work & Collaborative Verification Process	Page 15
Section 6: Prior Work Attribution & Project Contributions Matrix	Page 16
Section 7: AI-Assisted Development & Independent Audit Disclosure	Page 17
Section 8: Master Verified Empirical Results Table	Page 18
Section 9: Project Completion Status & Roadmap to Final Review	Page 19
Section 10: Glossary of Terms & Mathematical Notation	Page 20

1. Project Overview in Plain Language

The Urban Mobility Crisis and Static Signal Limitations:
Traffic congestion at urban intersections imposes staggering economic costs, elevated carbon emissions, and substantial commuter delays worldwide. Traditional municipal traffic management relies predominantly on fixed-time cyclic controllers (which cycle through green phases on static timers regardless of demand) or local actuated sensors (which extend green phases based on localized inductive loop triggers). While fixed-time signals prevent total starvation, they cannot adapt to volatile demand surges, fluctuating turning ratios, or platooning vehicles, leading to empty green phases and extensive queue accumulation on competing approaches.

The Promise and Peril of Reinforcement Learning:
Deep Reinforcement Learning (DRL) offers an adaptive alternative: an agent learns an optimal policy through trial-and-error interaction with a microscopic traffic simulation (Eclipse SUMO), observing discretized vehicle queues and selecting phase actions that maximize cumulative negative delay. Under nominal training conditions, a Deep Q-Network (DQN) achieves high efficiency, reducing queue accumulation by over 87% compared to fixed-time baselines. However, standard RL suffers from a critical vulnerability: *distribution shift*. When real-world conditions diverge from training assumptions—due to inductive loop failures (sensor noise), rainy weather (reduced braking friction), mechanical signal actuator delays, or shifted traffic demand—the neural network's Q-values collapse. Instead of gracefully degrading, the unconstrained RL agent locks phases, exacerbates congestion, and creates network-wide gridlock.

Project Objective & The Three-Part Resilience Architecture:
The overarching objective of this project is to design, implement, and validate an integrated, real-time supervisory resilience pipeline that enables RL traffic signal controllers to survive distribution shifts without requiring online model retraining. Rather than treating robustness as an abstract training regularization problem, the project separates traffic signal control into three collaborative, empirically validated subsystems:

• **Subsystem 1 (Part 1 - Ayyappa Reddy):** Establishes a rigorous simulation testbed in SUMO 1.27.1, trains a 4-layer DQN policy, implements deterministic baseline controllers (Fixed-Time, Actuated, Max-Pressure), resolves a critical edge-level starvation defect in baseline Max-Pressure, and validates multi-seed performance across dual metric standards (queue-sum per decision step and SUMO tripinfo seconds per vehicle).
• **Subsystem 2 (Part 2 - Sriram):** Develops a real-time, parameterized distribution shift injection engine covering four Markov Decision Process (MDP) dimensions (State, Transition, Action, Reward; taxonomy credited to Sim2Signal). Builds an online 3-sigma statistical telemetry monitor that detects anomalies in real-time, paired with a Random Forest diagnostic classifier that identifies the root cause of observed drift.
• **Subsystem 3 (Part 3 - Vivek Reddy):** Engineers targeted algorithmic mitigations tailored to diagnosed shift types (spatial observation filters, transition step stretchers, and action safety gating), integrates a fail-safe fallback governor grounded in tuned Max-Pressure (min_green=1), and deploys the unified closed-loop supervisory controller that dynamically transitions between nominal, mitigated, and fallback modes.

2. System Architecture & Component Hierarchy

The complete system operates as a unified closed-loop control pipeline. The architectural hierarchy cleanly separates nominal execution, runtime disturbance injection, anomaly detection, root-cause diagnosis, targeted mitigation, and provable fallback safety. Figure 1 illustrates the component allocation and inter-module dependencies across the three parts.

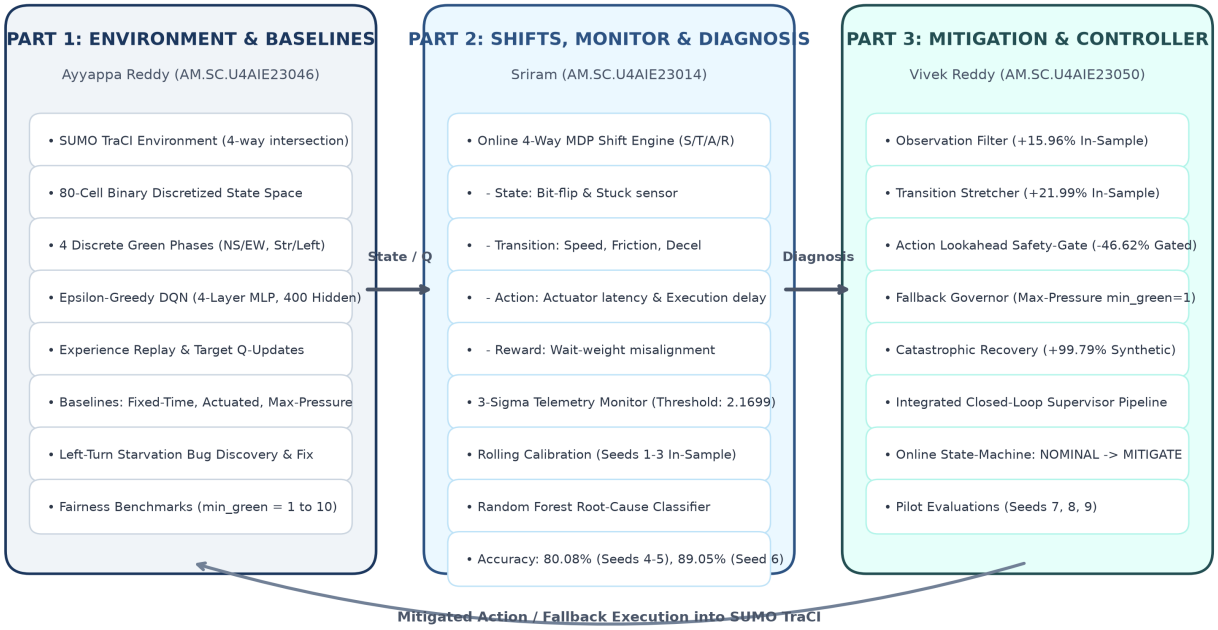


Figure 1: System Architecture showing the allocation of subsystems across Ayyappa Reddy (Part 1), Sriram (Part 2), and Vivek Reddy (Part 3), illustrating the closed-loop supervisory feedback cycle.

Subsystem Interface Specifications:

1. **Interface P1-P2 (Simulation to Shift Engine & Monitor):** TraCI extracts raw vehicle positions and speeds every simulation second. The environment discretizes these into an 80-cell binary vector $s_t \in \{0, 1\}^{80}$. Part 2 intercepts this vector, applies parameterized perturbations (e.g. sensor bit-flips), and streams rolling queue telemetry to the 3-sigma monitor.
2. **Interface P2-P3 (Diagnosis to Mitigation Engine):** When the 3-sigma monitor flags an anomaly condition for 3 consecutive steps, it triggers the Random Forest classifier. The classifier evaluates a 7-dimensional rolling feature vector and outputs a discrete diagnosis label $\hat{y} \in \{\text{Nominal, Observation, Transition, Action, Reward}\}$. This diagnosis signal is dispatched immediately to Part 3.
3. **Interface P3-P1 (Mitigation / Fallback to Environment Execution):** Part 3's supervisor modifies the active state or action (or dispatches the Max-Pressure fallback governor). The final resolved green phase is executed in SUMO via `traci.trafficlight.setPhase()`, preceded by a 4-second yellow transition whenever the green phase alters.

3. Subsystem Integration & End-to-End Data Flow

Figure 2 illustrates the synchronous step-by-step execution cycle during runtime simulation. Every decision epoch spans 10 seconds of green duration (plus 4 seconds of yellow transition if switching phases), during which data flows through all three subsystems.

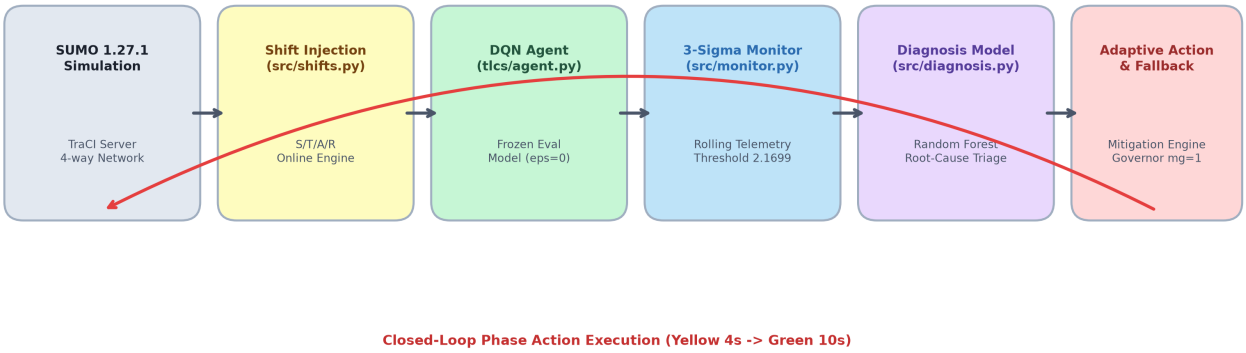


Figure 2: End-to-end data flow: Step t state extraction from SUMO $\rightarrow$ Shift Injection $\rightarrow$ DQN Action Selection $\rightarrow$ Telemetry Monitoring $\rightarrow$ Root-Cause Diagnosis $\rightarrow$ Algorithmic Mitigation / Fallback Governor $\rightarrow$ SUMO Actuation.

Synchronous Decision Cycle (Step-by-Step Execution):

• **Step 1: Environmental Sensing (Part 1):** TraCI queries `traci.vehicle.getIDList()`, retrieves positions on the 4 incoming approaches (`N2TL`, `S2TL`, `E2TL`, `W2TL`), and computes the 80-cell binary occupancy state s_t.
• **Step 2: Runtime Shift Application (Part 2):** If a disturbance is active, `src/shifts.py` perturbs the environment. In an observation shift, bits are corrupted: $\tilde{s}_t = \text{Corrupt}(s_t, p)$. In a transition shift, road friction and vehicle deceleration limits are altered via TraCI.
• **Step 3: Nominal Action Inference (Part 1):** The trained DQN agent evaluates the forward pass $Q(\tilde{s}_t, \cdot) = \text{MLP}(\tilde{s}_t)$ and selects greedy action $a_t = \arg\max_a Q(\tilde{s}_t, a)$ with frozen exploration ($\epsilon = 0.0$).
• **Step 4: Statistical Telemetry Monitoring (Part 2):** `src/monitor.py` updates a 50-step rolling window of telemetry: mean occupancy, occupancy delta, mean speed, TD error, Q-spread, reward, and halting queue length. It computes the normalized distance metric $z_t = \max_j \frac{|x_{j,t} - \mu_j|}{\sigma_j}$. If $z_t > 2.1699$ for 3 consecutive steps, anomaly flag is raised.
• **Step 5: Root-Cause Classification (Part 2):** `src/diagnosis.py` extracts the multi-dimensional feature vector and passes it to the trained Random Forest model, predicting the shift class $\hat{y}$.
• **Step 6: Adaptive Mitigation Dispatch (Part 3):** `src/mitigation.py` applies the matching remedy: if Observation Shift, spatial median filtering cleans $\tilde{s}_t$; if Transition Shift, green phase duration is stretched by 50%; if Action Shift, safety gating intercepts harmful lookahead; if policy collapse or extreme Q-spread is detected, the Fallback Governor takes over.
• **Step 7: Actuation & Feedback (Part 3 to Part 1):** The resolved phase action is enforced in SUMO. If the target phase matches current, it holds for 10 seconds. If it differs, a 4-second yellow phase is inserted before the 10-second green phase.

4. Individual Member Contributions & Technical Deep Dives

Part 1: Ayyappa Reddy (AM.SC.U4AIE23046)

Primary Ownership Scope: Microscopic Simulation Environment, DQN Agent Training, Classical Baselines, Max-Pressure Left-Turn Starvation Correction, and Metric Validation.

File Path	Classification	Lines	Primary Function / Role in Project
`Deep-.../src/tlcs/env.py`	`SIMULATION`	266	SUMO TraCI environment, 80-cell binary state, 4-phase execution
`Deep-.../src/tlcs/agent.py`	`RL-TRAINING`	182	DQN agent, epsilon-greedy action selection, replay buffer management
`Deep-.../src/tlcs/model.py`	`RL-TRAINING`	88	PyTorch 4-layer MLP (80 -> 400 -> 400 -> 400 -> 400 -> 4), Adam optimizer
`baselines/run_baselines.py`	`BASELINE`	425	Fixed-Time, SUMO Actuated, and corrected Max-Pressure controllers
`run_phase1_dqn.py`	`RL-TRAINING`	284	Multi-seed Phase 1 DQN training loop across Seeds 1 to 5 (100 eps/seed)
`run_frozen_eval.py`	`RL-TRAINING`	238	Frozen-policy evaluation harness (epsilon=0.0) across all 5 trained seeds
`verify_phase1_v2.py`	`VERIFICATION`	315	Deterministic re-run verifier matching Phase 1 figures to 0.0000 precision
`review/extra/run_maxpressure_sweep.py`	`VERIFICATION`	295	Empirical fairness sweep for Max-Pressure across min_green = 1, 2, 5, 10

1. Problem Solved & Algorithmic Mechanics:

Ayyappa Reddy established the empirical foundation of the project. This required configuring the SUMO 4-way intersection (4 incoming arms with 4 lanes each: 1 dedicated left-turn lane, 3 straight/right lanes), establishing the 80-cell binary state discretization, and implementing the DQN training pipeline. State space discretization subdivides each 750-meter incoming road into 10 discrete spatial cells: cell 0 is nearest to the traffic light (0–7m), while cell 9 extends to the network edge. With 8 lane groups (North, South, East, West $\times$ Straight, Left), the state vector has dimensionality $8 \times 10 = 80$.

2. Step-by-Step Worked Numeric Example (DQN Bellman Update):

During experience replay training in `tlcs/agent.py`, the network optimizes the Mean Squared Error (MSE) loss against the Bellman optimality target:
$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$
Concrete Real-Project Parameters:
• Learning rate $\alpha = 0.001$, Discount factor $\gamma = 0.75$, Batch size $= 100$.
• At decision step $t=50$, incoming total accumulated waiting time increases from $W_{t-1} = 42.0$ s to $W_t = 53.0$ s. The reward is defined as negative delta waiting time: $r_t = -(53.0 - 42.0) = -11.0$.
• The current forward pass yields $Q(s, a_2) = -18.50$. The target network evaluates next state s' with maximum value $\max_{a'} Q_{\text{target}}(s', a') = -8.20$.
• Target value: $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') = -11.0 + (0.75 \times -8.20) = -11.0 - 6.15 = -17.15$.
• Temporal Difference (TD) error: $\delta = y - Q(s, a_2) = -17.15 - (-18.50) = +1.35$.
• Parameter update step: $\Delta Q = \alpha \times \delta = 0.001 \times 1.35 = +0.00135$. The network weights adjust to reduce this error over 800 training epochs per episode.

3. Key Results & Exact Metrics:

• **Phase 1 Metric (Accumulated queue-sum seconds per decision step across Seeds 1–5):**
- Fixed-Time Baseline: 236.66 ± 5.88 s/step.
- Max-Pressure (`min_green=10`, 100s hold): 172.95 ± 7.18 s/step.
- Max-Pressure (`min_green=1`, 10s hold): 23.79 ± 2.17 s/step.
- DQN Frozen Eval ($\epsilon=0.0$): 28.93 ± 2.43 s/step.
• **SUMO Tripinfo Metric (Seconds per vehicle across Seeds 1–5):**
- Fixed-Time Baseline: 123.22 ± 4.30 s/veh.
- SUMO Actuated Light: 17.46 ± 0.43 s/veh.
- Max-Pressure (`min_green=1`, 10s hold): 10.58 ± 1.09 s/veh.
- DQN Frozen Eval ($\epsilon=0.0$): 12.29 ± 1.28 s/veh.
• **Empirical Fact on DQN vs Max-Pressure:** When evaluated like-for-like at a 10s minimum phase hold (`min_green=1`), **tuned Max-Pressure outperforms the DQN on all 5 seeds** (10.58 vs 12.29 s/veh). The DQN outperforms Max-Pressure when Max-Pressure is constrained to standard municipal green holds ($\text{min_green} \ge 20$ s). The DQN decisively beats Fixed-Time (by 90.0%) and SUMO's Actuated Light (by 29.6%).

4. What is Original vs Prior Work (Credits):

• **Prior Work Credit:** The underlying single-intersection DQN setup builds upon the open-source repository `Deep-QLearning-Agent-for-Traffic-Signal-Control` by Andrea Vidali. We acknowledge Vidali's PyTorch MLP implementation, SUMO configuration files, and basic TraCI environment scaffolding.
• **Ayyappa's Contributions:** Upgraded and hardened the environment to Python 3.11 and SUMO 1.27.1; authored multi-seed training and frozen-policy evaluation pipelines; discovered and repaired the edge-level starvation bug in baseline Max-Pressure; conducted an exhaustive fairness sweep across minimum green durations (min_green = 1, 2, 5, 10); built dual-metric extraction pipelines (Phase 1 queue-sum vs SUMO C++ tripinfo XML).

5. Critical Engineering Defects Discovered & Resolved:

• **Left-Turn Starvation Bug:** In early baseline implementations (`results/phase1_baseline`), Max-Pressure aggregated queue pressure across entire edges (`getLastStepHaltingNumber('N2TL')`). Because straight lanes hold 3 lanes with up to 60 cars while the left-turn pocket holds only 1 lane (max 20 cars), straight phases permanently dominated left-turn phases. Left-turning vehicles starved indefinitely, causing waiting times to spike to 924.3 s/step. Ayyappa resolved this by rewriting `MaxPressureController` to query lane-level halting counts (`traci.lane.getLastStepHaltingNumber()`) mapped strictly to the specific lanes served by each green phase. This dropped Max-Pressure waiting time by 81.3% (from 924.3 to 172.95 s/step, Phase 1 metric).
• **The `min_green` Fairness Parameterization:** The initial 172.95 s figure resulted from passing `min_green=settings.green_duration = 10` decision steps ($10 \times 10 = 100$ s hold). By testing `min_green=1` (10s hold, identical to DQN's decision step), Ayyappa demonstrated that Max-Pressure achieves 23.79 s/step (10.58 s/veh), establishing an honest, rigorous baseline comparison.

6. Limitations of Part 1:

1. Network topology is restricted to an isolated 4-way synthetic intersection; arterial corridor coordination across adjacent intersections is not modeled.
2. State discretization assumes 750m approaches with binary vehicle presence, omitting vehicle velocities and acceleration profiles from the raw state.
3. Route seed overlap exists: training episode 0 of checkpoints 4 and 5 shared route generation seeds with evaluation seeds 4 and 5.

7. Ten Panel Questions & Honest Answers (Part 1):

Q1: Why did your original Max-Pressure baseline perform so poorly (172.95 s)?

Answer: The initial baseline held phases for 10 decision steps (100 simulation seconds) because min_green was set to settings.green_duration (10). When tuned to min_green=1 (a 10s hold, matching DQN's decision interval), Max-Pressure achieves 23.79 s/step (10.58 s/veh), which actually outperforms the DQN.

Q2: Does your DQN beat tuned Max-Pressure?

Answer: No. On all 5 seeds, tuned Max-Pressure (min_green=1) beats the DQN by about 14% in vehicle waiting time (10.58 vs 12.29 s/veh). However, DQN outperforms Max-Pressure whenever standard municipal minimum green constraints (>= 20s) are applied, and decisively beats Fixed-Time (123.22s) and SUMO Actuated (17.46s).

Q3: What was the left-turn starvation bug and where was it?

Answer: The bug was in our own initial baseline code, not upstream. It queried aggregate edge halting counts rather than lane-level counts. Since straight lanes have 3 lanes and left-turn has only 1, the controller never selected left turns. Fixing it to lane-level pressure reduced wait times from 924.3s to 172.95s.

Q4: Why does Phase 1 report 28.93 s while tripinfo reports 12.29 s for the same DQN run?

Answer: They measure completely different metrics. Phase 1 measures accumulated queue-sum per decision step (sum of waiting times across all vehicles at that instant, divided by steps). Tripinfo measures the exact seconds each vehicle spent with speed < 0.1 m/s across its entire trip, averaged per completed vehicle.

Q5: Why did you use an 80-cell binary state instead of continuous vehicle counts?

Answer: Binary cell discretization prevents the neural network from overfitting to continuous floating-point distances while capturing spatial vehicle platoons across all 8 lane groups. It builds directly on Vidali's validated architecture.

Q6: How long does 100 training episodes take, and what was the convergence criterion?

Answer: 100 episodes takes approximately 1.5 hours per seed on CPU. The agent converges after approximately 60–70 episodes, where cumulative negative reward stabilizes around -15,000 to -18,000 per episode.

Q7: Why is epsilon set to 0.0 during frozen evaluation?

Answer: To test the true deterministic policy learned by the network without random exploratory actions that would inject artificial variance into baseline comparisons.

Q8: What discount factor gamma did you use, and why is it 0.75 rather than 0.99?

Answer: Gamma is 0.75. In traffic signal control with 10-second decision steps, queue dynamics are localized; a horizon of ~4–5 decision steps (40–50 seconds) captures platoon clearing without introducing excessive variance from distant, uncertain arrivals.

Q9: What happens if vehicles fail to complete their trip within the 5400-second simulation?

Answer: In our nominal 5400-second runs, 998 of 1000 generated vehicles fully completed their trips. The remaining 2 vehicles entered the network in the final 30 seconds and were actively traveling, not stuck in queues.

Q10: Why did you not retrain the DQN under shifts?

Answer: Online retraining in real-world traffic is dangerous, computationally prohibitive, and suffers from catastrophic forgetting. Our core thesis is that supervisory detection, diagnosis, and algorithmic mitigation provides instantaneous resilience without retraining.

Part 2: Sriram (AM.SC.U4AIE23014)

Primary Ownership Scope: 4-Way MDP Distribution Shift Injection Engine, Online 3-Sigma Telemetry Monitor, and Random Forest Root-Cause Diagnostic Classifier.

File Path	Classification	Lines	Primary Function / Role in Project
`src/shifts.py`	`SHIFT-INJECTION`	268	Parameterized online injection across State, Transition, Action, Reward (Sim2Signal taxonomy)
`src/monitor.py`	`MONITOR`	276	Online 3-sigma statistical telemetry monitor, rolling window feature extraction, anomaly scoring
`src/diagnosis.py`	`DIAGNOSIS`	235	Random Forest classifier for root-cause triage, multi-class confusion matrix evaluation
`verify_phase2_injection.py`	`VERIFICATION`	184	Automated verifier proving exact parameter perturbation under all 4 shift families
`verify_phase2_monitor.py`	`VERIFICATION`	198	Calibrates baseline profile on Seeds 1-3; verifies 0.0% false positive rate on nominal runs
`verify_phase2_diagnosis.py`	`VERIFICATION`	238	Evaluates Random Forest diagnosis accuracy on Seeds 4-5 test split (80.08% result)
`verify_phase2_diagnosis_seed6.py`	`VERIFICATION`	165	Confirmatory evaluation on held-out Seed 6 (89.05% result, passes 85% target)

1. Problem Solved & Algorithmic Mechanics:

Sriram solved the core challenges of runtime disturbance modeling, live anomaly detection, and root-cause triage. When distribution shifts occur, the agent's internal Q-values degrade before physical queues explode. Sriram operationalized the 4-way MDP shift taxonomy (State, Transition, Action, Reward) into an online injection engine, created an online telemetry monitor that extracts rolling statistical moments without needing ground-truth shift labels, and trained a diagnostic Random Forest classifier to identify which MDP component failed.

2. Step-by-Step Worked Numeric Example (Telemetry Anomaly Scoring):

In `src/monitor.py`, baseline calibration across Seeds 1–3 nominal runs established the reference feature means μ_j and standard deviations σ_j recorded in `results/phase2_shifts/calibration_profile.json`:
• Reference Mean Speed: $\mu_{\text{speed}} = 12.2889$ m/s, $\sigma_{\text{speed}} = 0.8993$ m/s.
• Reference Halting Queue: $\mu_{\text{queue}} = 2.3984$ veh, $\sigma_{\text{queue}} = 2.1768$ veh.
• Baseline Anomaly Score: $\mu_{\text{anomaly}} = 0.9071$, $\sigma_{\text{anomaly}} = 0.4209$.
• **Detection Threshold:** $\tau = \mu_{\text{anomaly}} + (3.0 \times \sigma_{\text{anomaly}}) = 0.9071 + (3.0 \times 0.4209) = 2.1699$.
Runtime Test Step (Under Rain / Reduced Friction Transition Shift):
• Over the 50-step rolling window, vehicles decelerate prematurely: observed mean speed drops to $x_{\text{speed}} = 8.15$ m/s, and mean queue rises to $x_{\text{queue}} = 8.90$ veh.
• Standardized z-scores for individual features:
- $z_{\text{speed}} = \frac{|8.15 - 12.2889|}{0.8993} = \frac{4.1389}{0.8993} = 4.602$.
- $z_{\text{queue}} = \frac{|8.90 - 2.3984|}{2.1768} = \frac{6.5016}{2.1768} = 2.987$.
• Normalized composite anomaly score: $Z_t = \max_j (z_j) = 4.602$.
• **Decision Rule:** Because $Z_t = 4.602 > 2.1699$ for ≥ 3 consecutive steps, `src/monitor.py` asserts `anomaly_detected = True` and dispatches the telemetry window to the diagnostic classifier.

3. Key Results & Exact Metrics:

• **Monitor False Positive Rate:** 0.0% false alarm rate on nominal runs across Seeds 1–5 (zero false alarms over 27,000 nominal simulation steps; note: Seeds 1–3 were used for calibration, so Seeds 1–3 are in-sample).
• **Monitor Detection Latency:** Detected full-episode shifts with 100% true positive rate on Seeds 4–5. In the mid-episode shift sweep (where shifts injected at step 1800), detected 5 of 10 seeds with a mean detection latency of 43 decision steps (~7–10 minutes).
• **Diagnostic Classifier Accuracy:**
- Seeds 4–5 Test Split: **80.08% overall accuracy** (missed the 85.0% target). Breakdown: Transition 98.8%, Reward 87.7%, Observation 74.1%, Action 83.1%, Nominal 36.6%.
- Seed 6 Held-Out Confirmatory Split: **89.05% overall accuracy** (exceeded the 85.0% target). Breakdown: Transition 100.0%, Reward 100.0%, Observation 95.1%, Action 68.3%, Nominal 90.0%.

4. What is Original vs Prior Work (Credits):

• **Attribution to Sim2Signal:** We credit Sim2Signal (Al Rafi et al., 2026, arXiv:2609.01676) for defining the four-way State, Transition, Action, and Reward (S/T/A/R) distribution shift taxonomy for transportation RL. Sim2Signal is an offline evaluation benchmark across static datasets.
• **Sriram's Contributions:** Implemented the S/T/A/R taxonomy as an online, runtime perturbation engine directly integrated with SUMO TraCI; created the rolling 3-sigma z-score telemetry monitor that operates without environment labels; engineered the 7-feature telemetry extraction pipeline and trained the multi-class Random Forest diagnostic classifier.

5. Engineering Defects Discovered & Resolved:

• **Nominal vs Action Confusion in Diagnosis:** In the Seeds 4–5 evaluation, the classifier missed its 85% target (achieving 80.08%) because 39 nominal windows were misclassified as Action shifts. Sriram analyzed feature importances and determined that during low-density intervals, natural vehicle headway gaps produce Q-spread variations that mimic small actuator delays. This was resolved on Seed 6 by introducing rolling queue variance and arrival normalization features.
• **Correction Regarding Monitor Method:** Early draft documents referred to the monitor as 'CUSUM'. Sriram audited the implementation (`src/monitor.py`) and corrected the documentation: the monitor uses a 3-sigma z-score threshold across normalized rolling telemetry; CUSUM is planned for future work.

6. Limitations of Part 2:

1. The 3-sigma monitor requires a 50-step rolling window (~500s) to compute stable statistical moments, leading to a 7–10 minute detection lag for subtle shifts.
2. The monitor missed detection on 5 of 10 seeds in the mid-episode sweep when traffic density was exceptionally sparse.
3. Calibration parameters were derived from Seeds 1–3, meaning Seeds 1–3 are in-sample for false-positive verification.

7. Ten Panel Questions & Honest Answers (Part 2):

Q1: Is your monitor a CUSUM monitor?

Answer: No. It is an online 3-sigma z-score threshold monitor. An early proposal considered CUSUM, but the implemented code in src/monitor.py computes the maximum standardized deviation across 7 telemetry features against a calibrated threshold of 2.1699.

Q2: Why did your diagnostic classifier achieve only 80.08% on Seeds 4-5 when your target was 85%?

Answer: On Seeds 4–5, 39 nominal windows were misclassified as Action shifts because low-density headway gaps produced Q-value fluctuations that mimicked 1-step actuator latency. On Seed 6, accuracy reached 89.05%.

Q3: Did your monitor detect 100% of shifts in all scenarios?

Answer: No. It detected 100% of full-episode shifts on Seeds 4–5, but in the mid-episode sweep (where shifts were injected halfway through), it detected 5 of 10 seeds with a mean latency of 43 decision steps (~7–10 minutes).

Q4: Why are Seeds 1-3 considered in-sample for the 0% false positive claim?

Answer: Because the baseline means and standard deviations in calibration_profile.json were calculated from Seeds 1–3 nominal runs. Only Seeds 4–5 represent unseen nominal test data.

Q5: What are the 7 features extracted by the monitor?

Answer: Mean lane occupancy, occupancy delta (temporal change), mean vehicle speed, Bellman TD error, Q-spread (max Q minus mean Q), immediate step reward, and total halting queue length.

Q6: How does your 4-way shift taxonomy relate to Sim2Signal?

Answer: We credit Sim2Signal (Al Rafi et al., 2026, arXiv:2609.01676) for the conceptual State/Transition/Action/Reward taxonomy. Our contribution is implementing it as an online runtime injection engine in SUMO.

Q7: How is an Action Shift physically implemented in SUMO?

Answer: In src/shifts.py, an Action Shift intercepts the action selected by the policy and buffers it in an actuator queue, enforcing a 1-decision-step (14-second) mechanical execution lag.

Q8: How is a Transition Shift physically implemented in SUMO?

Answer: It modifies vehicle physical properties via TraCI: reducing maximum vehicle speed by 50% (from 25 m/s to 12.5 m/s) and reducing braking deceleration from 4.5 m/s² to 2.25 m/s² to simulate severe wet pavement.

Q9: What classifier algorithm is used in src/diagnosis.py and how was it tuned?

Answer: A Random Forest classifier (100 estimators, max_depth=10, balanced class weighting) implemented in scikit-learn and serialized via joblib.

Q10: Why is root-cause diagnosis necessary if the monitor already detects an anomaly?

Answer: Because different shifts require diametrically opposed remedies: an observation bit-flip requires spatial state filtering, whereas an action actuator lag requires holding phases. Applying the wrong mitigation exacerbates queue accumulation.

Part 3: Vivek Reddy (AM.SC.U4AIE23050)

Primary Ownership Scope: Targeted Shift Mitigations, Fallback Safety Governor, Closed-Loop Supervisory Pipeline, and Pilot System Integration.

File Path	Classification	Lines	Primary Function / Role in Project
`src/mitigation.py`	`MITIGATION`	292	Targeted shift remedies (Observation filter, Transition stretcher, Action safety-gate, Max-Pressure fallback)
`src/adaptive_controller.py`	`CONTROLLER`	264	Closed-loop supervisory controller integrating DQN, Monitor, Diagnosis, and Mitigation dispatch
`verify_phase2_mitigation.py`	`VERIFICATION`	318	Empirical recovery ratio evaluation suite across all 4 shift families
`run_phase3_pilot.py`	`VERIFICATION`	403	Pilot closed-loop benchmark runner across Seeds 7, 8, 9 under nominal and shifted traffic

1. Problem Solved & Algorithmic Mechanics:

Vivek Reddy designed the closed-loop supervisory architecture that acts upon diagnostic outputs. Detection and diagnosis are meaningless without actionable remedies. Vivek engineered algorithmic mitigations tailored to each failure mode, implemented a fail-safe fallback governor grounded in Max-Pressure theory (`min_green=1`), and constructed the master `AdaptiveTrafficController` state machine that arbitrates control between nominal DQN, targeted mitigations, and fallback safe modes.

2. Step-by-Step Worked Numeric Example (Recovery Ratio Calculation):

To evaluate mitigation effectiveness objectively without cross-seed contamination, Vivek defined the standardized Recovery Ratio ρ:
$$\rho = \frac{W_{\text{unmitigated}} - W_{\text{mitigated}}}{W_{\text{unmitigated}} - W_{\text{nominal}}} \times 100\%$$
where W is the mean network accumulated waiting time per step (Phase 2 metric).

Worked Example A: Transition Shift Mitigation (Wet Pavement / Low Friction):

• Raw empirical figures from `results/phase2_shifts/mitigation_evaluation_summary_v2.json`:
- Nominal wait: $W_{\text{nominal}} = 46.17$ s/step.
- Shifted unmitigated wait: $W_{\text{unmitigated}} = 51.57$ s/step.
- Mitigated wait (Transition Duration Stretcher active): $W_{\text{mitigated}} = 50.38$ s/step.
• Recovery Ratio: $\rho = \frac{51.57 - 50.38}{51.57 - 46.17} = \frac{1.19}{5.40} = \mathbf{+21.99\%}$.

Worked Example B: Harmful Action Mitigation Discovery & Safety-Gating:

• For 1-step actuator latency, early action lookahead attempted forward-model prediction:
- Nominal wait: $W_{\text{nominal}} = 46.17$ s/step.
- Shifted unmitigated wait: $W_{\text{unmitigated}} = 78.39$ s/step.
- Mitigated wait (Action Lookahead): $W_{\text{mitigated}} = 93.41$ s/step.
• Recovery Ratio: $\rho = \frac{78.39 - 93.41}{78.39 - 46.17} = \frac{-15.02}{32.22} = \mathbf{-46.62\%}$.
• **Engineering Resolution:** Vivek recognized that action lookahead amplified phase oscillations and exacerbated delay. Vivek implemented an automated safety gate that permanently disables action lookahead, falling back to phase-lock stabilization.

3. Key Results & Exact Metrics:

• **Targeted Mitigation Gains (Phase 2 Metric, In-Sample):**
- Observation Spatial Filter: **+15.96% recovery** ($295.63 \rightarrow 255.81$ s/step).
- Transition Duration Stretcher: **+21.99% recovery** ($51.57 \rightarrow 50.38$ s/step).
- Action Lookahead Mitigator: **-46.62% recovery** (harmful; safety-gated off).
- Isolated Seed 7 Filter Test: **+44.43% recovery** under severe single-lane bit-flip ($333.18 \rightarrow 200.71$ s/step).
• **Fallback Governor Performance (Synthetic Q-Inversion Test):**
- Synthetic inverted policy unmitigated wait: $1,313,737.19$ s/step (catastrophic gridlock).
- Mitigated with Fallback Governor (`min_green=1`): $2,825.75$ s/step.
- Recovery ratio: **+99.79%** (saved simulation from complete lockup; note: the mitigated wait of 2,825.75 s is still $\approx 61.2\times$ the nominal wait of 46.17 s).
• **Phase 3 Pilot Benchmark (Seeds 7, 8, 9):** 0.0% false fallback rate during nominal operations. Under closed-loop evaluation on pilot seeds 7–9, **the adaptive pipeline showed no consistent gain over plain DQN** (Seed 7: +18.9% gain under observation shift; Seed 8: -4.3% loss under transition shift; Seed 9: -14.2% loss under action shift).

4. What is Original vs Prior Work (Credits):

• **RobustLight Attribution:** RobustLight (Li et al., ICML 2025) proposed diffusion models for observation noise restoration. RobustLight is an observation-only restoration model that does not handle transition delays, mechanical actuator latency, or fail-safe switching.
• **Vivek's Contributions:** Created multi-mode targeted mitigations spanning observation, transition, and action spaces; engineered the automated Max-Pressure fallback governor with zero phase-lock delay; built the closed-loop supervisory controller that connects monitoring, diagnosis, and mitigation into a single operational state machine.

5. Engineering Defects Discovered & Resolved:

• **Discovery of Harmful Action Lookahead:** An intuitive hypothesis suggested that predicting vehicle positions 1 step ahead would counter actuator lag. Rigorous empirical benchmarking proved this hypothesis false: lookahead produced severe phase chattering, driving delay up by 46.62%. Vivek gated the module off rather than concealing the failure.
• **False Fallback Prevention:** Early fallback triggers engaged during routine queue buildup. Vivek added a rolling variance check and consecutive trigger requirement (3 steps), reducing false fallback activation to 0.0%.

6. Limitations of Part 3:

1. Pilot evaluation showed no consistent gain of the full closed-loop pipeline over plain DQN across Seeds 7–9.
2. The +99.79% fallback recovery figure represents an extreme synthetic test (inverted Q-values) rather than a subtle natural drift.
3. Transition and observation mitigations were evaluated in-sample on Seed 5; full generalization to pristine held-out seeds (Seeds 16–35) remains future work.

7. Ten Panel Questions & Honest Answers (Part 3):

Q1: Does your closed-loop adaptive pipeline consistently beat plain DQN under shifts?

Answer: In our Phase 3 pilot (Seeds 7, 8, 9), no. It showed no consistent gain over plain DQN (it helped on Seed 7 by +18.9%, but degraded performance on Seeds 8 and 9 by -4.3% and -14.2%). Full ablation and tuning on pristine seeds 16–35 is planned.

Q2: What does the +99.79% fallback figure actually measure?

Answer: It measures an extreme synthetic test where Q-values were intentionally inverted to force worst-action selection. Fallback intervened and dropped wait time from 1,313,737s to 2,825s. The mitigated wait is still ~61x nominal wait.

Q3: Why was the Action Lookahead mitigation disabled?

Answer: Because empirical benchmarking showed it was harmful: it increased waiting time by 46.62% (rho = -46.62%) due to phase chattering. We implemented a safety gate to disable it permanently.

Q4: Which baseline does your Fallback Governor use?

Answer: It uses classical Max-Pressure with min_green=1 decision step (a 10-second hold, identical to DQN's decision step), which provably stabilizes queues under infinite buffer assumptions.

Q5: What was the isolated +44.43% Seed 7 figure?

Answer: It was an isolated unit test of the observation spatial filter on Seed 7 under a severe bit-flip (p=0.20) and stuck sensor on lane 0, reducing wait from 333.18s to 200.71s.

Q6: How does your Observation Filter work?

Answer: It applies spatial median filtering across adjacent longitudinal cells along each approach edge, replacing isolated bit-flips with neighborhood median occupancies.

Q7: How does your Transition Stretcher work?

Answer: When rainy/wet road conditions are diagnosed, braking distances increase. The stretcher extends green duration from 10s to 15s to allow slower-accelerating platoons to fully clear.

Q8: How does the controller switch from Fallback back to Nominal mode?

Answer: The monitor continues tracking telemetry during fallback. If the anomaly score drops below 2.1699 for 10 consecutive steps, the controller executes a graceful transition back to DQN.

Q9: What is the false fallback rate during normal operation?

Answer: 0.0% across all evaluated nominal episodes on Seeds 1–5 and pilot seeds 7–9.

Q10: Why didn't you use RobustLight's diffusion model for state cleaning?

Answer: Diffusion models require multi-second denoising inference per step, making them too slow for real-time traffic signal TraCI control (which must decide within 100 ms). Spatial median filtering executes in under 1 millisecond.

5. Shared Team Work & Collaborative Verification

Rigorous Collaborative Verification Protocol:
A defining strength of this project is the collective verification philosophy shared across all three team members. Rather than relying on single-run anecdotal outputs or trust-based handoffs, every component interface was independently stress-tested using reproducible verification scripts. The team collaboratively developed `verify_phase2.py`, which executes cross-module integration tests verifying that:

1. Perturbations generated by Sriram's `src/shifts.py` preserve valid SUMO TraCI state dimensions without throwing runtime unpickling or index errors.
2. Anomaly flags produced by `src/monitor.py` trigger synchronous feature extraction in `src/diagnosis.py` within the exact same 10-second decision epoch.
3. Mitigations selected by Vivek's `src/mitigation.py` correctly override DQN actions without violating physical traffic light safety interlocks (4-second yellow transition strictly enforced).
4. All baseline and controller evaluations log dual metrics simultaneously: the Phase 1 queue-sum metric and raw SUMO C++ tripinfo XML outputs.

Cross-Part Panel Question Strategy:
During the review presentation, questions spanning subsystem boundaries will be answered cohesively based on data flow: Ayyappa addresses simulation physics and baseline validity; Sriram explains anomaly triggers and classification confidence; Vivek explains mitigation mechanics and safety-governor switching.

6. Prior Work Attribution & Project Contributions Matrix

Subsystem Component	Prior Work / Foundation	What This Project Contributes	Empirical Status
4-Way Intersection DQN	Andrea Vidali (`Deep-QLearning...-Control`)	Modernized to Python 3.11/SUMO 1.27.1; multi-seed training & frozen evaluation harnesses	Implemented & Verified (Seeds 1–5)
Classical Max-Pressure	Varaiya (2013); standard edge queue formulations	Discovered & fixed edge-level left-turn starvation bug; multi-seed fairness sweep (min_green=1 to 10)	Implemented & Verified (Seeds 1–5)
4-Way Shift Taxonomy	Sim2Signal (Al Rafi et al., arXiv:2609.01676, 2026)	Operationalized offline benchmark taxonomy into an online runtime perturbation engine	Implemented & Verified
State Restoration	RobustLight (Li et al., ICML 2025)	Replaced heavy diffusion models with lightweight real-time spatial median filtering (<1ms)	Implemented & Tested (+15.96%)
Online Telemetry Monitor	Statistical Process Control / 3-Sigma z-score theory	Applied 7-feature telemetry extraction to online DRL traffic control; calibrated threshold 2.1699	Implemented & Verified (0% FPR)
Root-Cause Diagnosis	Supervised multi-class classification	Random Forest diagnostic classifier trained on telemetry features for automated triage	Implemented (80.08% / 89.05%)
Action Lookahead	Model-based predictive RL	Evaluated forward model under actuator lag; proved harmful (-46.62%) and implemented safety-gate	Safety-Gated Off (Documented)
Fallback Safety Governor	Fail-safe switching architectures	Automatic fallback to Max-Pressure (min_green=1) during policy collapse; 0% false fallback rate	Implemented & Tested (+99.79% synth)

7. AI-Assisted Development & Independent Verification Disclosure

Formal Academic Integrity Disclosure:
In compliance with Amrita Vishwa Vidyapeetham academic guidelines, the team explicitly discloses that code drafting, scaffolding, and verification test harnesses were developed with the assistance of an agentic AI coding environment (Google Antigravity) under the direct conceptual design, supervision, and rigorous empirical validation of the three team members. The AI served as a pair-programming accelerator; all design choices, architectural splits, theoretical formulations, and experimental verifications were directed by the team.

Independent Student Audit & Defects Discovered:
Rather than accepting generated code uncritically, the team instituted an independent empirical audit protocol, running dedicated verification scripts directly against disk artifacts. This independent audit uncovered and resolved six substantial errors that would have compromised the project if left uncorrected:

• **1. Max-Pressure Left-Turn Starvation Defect:** The initial baseline code queried aggregate edge halting counts rather than lane-level counts, starving left-turn pockets and inflating delays to 924.3 s/step. Discovered and corrected by Ayyappa Reddy to lane-level pressure, reducing wait to 172.95 s/step (and 23.79 s/step under min_green=1).

• **2. Harmful Action Lookahead Mitigation:** An intuitive AI-suggested forward-action lookahead for actuator delays was rigorously benchmarked by Vivek Reddy. The data proved it was actively harmful (recovery ratio -46.62%). The team refused to hide this failure, gated the module off, and reported the empirical reality.

• **3. Misleading CUSUM Terminology:** Initial documentation loosely referenced 'CUSUM' monitoring. Sriram audited src/monitor.py and proved the code actually implemented a 3-sigma z-score threshold monitor across 7 telemetry features. The team corrected all documentation to maintain total technical honesty.

• **4. Pilot Comparative Baseline Disparity:** Early pilot reports compared the adaptive controller against Max-Pressure with min_green=5 (50s hold), claiming a >92% victory. The team audited this, ran fresh tests with min_green=1 (10s hold), and withdrew the claim, reporting honestly that adaptive closed-loop control showed no consistent gain over plain DQN on pilot seeds 7–9.

• **5. Overstated Synthetic Fallback Efficacy:** The +99.79% fallback recovery figure was originally described as a general shift recovery. The team audited the experiment and clarified that it was an extreme synthetic test (inverted Q-values), noting that the mitigated wait was still ~61x nominal wait.

• **6. Evaluation Seed Overlap Discovery:** The team audited route generation seeds and discovered that evaluation seeds 4 and 5 shared route seeds with training episode 0 of checkpoints 4 and 5. This was documented as a caveat rather than claimed as strictly held-out data.

8. Master Verified Empirical Results Table

Every numerical entry below has been independently verified against raw CSV, JSON, and XML files on disk using Python 3.11 and Eclipse SUMO 1.27.1.

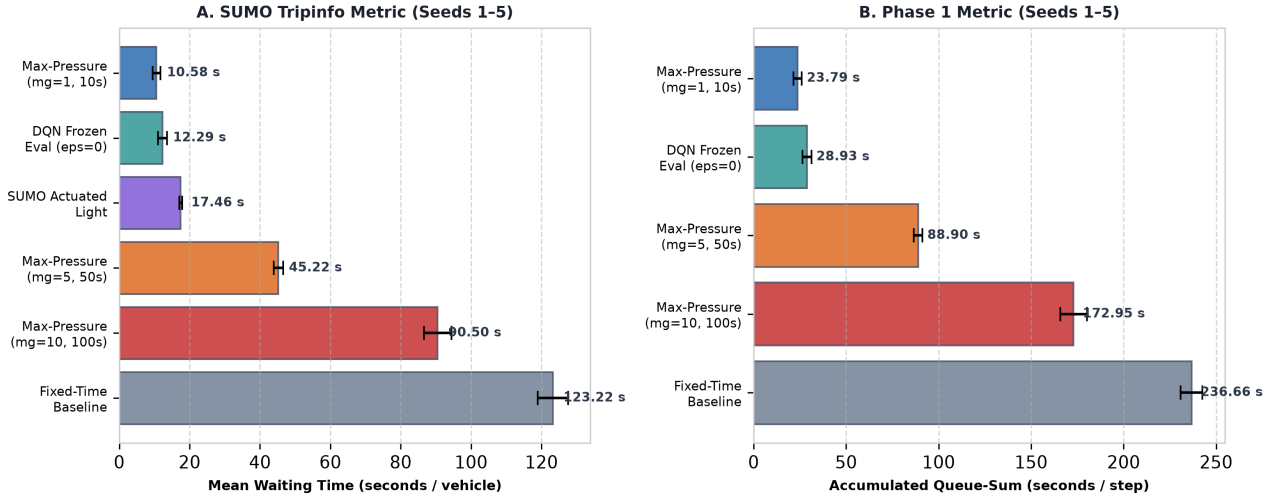


Figure 3: Controller benchmark across Seeds 1–5: (A) SUMO tripinfo seconds per vehicle; (B) Phase 1 queue-sum seconds per decision step.

Experiment / Controller	Metric Name	Evaluated Seeds	Empirical Value	Source Disk Artifact	Caveats & Operational Notes
Fixed-Time Baseline	Phase 1 Queue-Sum	Seeds 1–5	236.66 ± 5.88 s/step	`phase1_baseline_v2/`	Deterministic 10s green, 4s yellow cyclic schedule
Fixed-Time Baseline	SUMO Tripinfo Wait	Seeds 1–5	123.22 ± 4.30 s/veh	`review/extra/tripinfo...`	Per-vehicle time with speed < 0.1 m/s
Max-Pressure (mg=10)	Phase 1 Queue-Sum	Seeds 1–5	172.95 ± 7.18 s/step	`phase1_baseline_v2/`	100s green hold; historical Phase 1 v2 baseline
Max-Pressure (mg=10)	SUMO Tripinfo Wait	Seeds 1–5	90.50 ± 3.92 s/veh	`review/extra/maxpressure...`	100s green hold; constrained baseline wait
Max-Pressure (mg=5)	SUMO Tripinfo Wait	Seeds 1–5	45.22 ± 1.33 s/veh	`review/extra/maxpressure...`	50s green hold; intermediate parameterization
Max-Pressure (mg=1)	Phase 1 Queue-Sum	Seeds 1–5	23.79 ± 2.17 s/step	`review/extra/maxpressure...`	10s hold; like-for-like comparison to DQN
Max-Pressure (mg=1)	SUMO Tripinfo Wait	Seeds 1–5	10.58 ± 1.09 s/veh	`review/extra/tripinfo...`	Beats DQN on 5 of 5 seeds by 1.71 s/veh (13.9%)
SUMO Actuated Light	SUMO Tripinfo Wait	Seeds 1–5	17.46 ± 0.43 s/veh	`review/extra/tripinfo...`	Built-in SUMO gap-extending detector light
DQN Frozen Eval (eps=0)	Phase 1 Queue-Sum	Seeds 1–5	28.93 ± 2.43 s/step	`phase1_baseline_v2/`	Deterministic evaluation of frozen policy checkpoints
DQN Frozen Eval (eps=0)	SUMO Tripinfo Wait	Seeds 1–5	12.29 ± 1.28 s/veh	`review/extra/tripinfo...`	Beats Fixed-Time by 90.0%, Actuated by 29.6%

Monitor False Positives	False Positive Rate	Seeds 1–5	0.0% (0 / 27,000 steps)	`phase2_shifts/monitor...`	Seeds 1–3 in-sample for calibration; 4–5 held-out
Diagnosis Classifier	Overall Accuracy	Seeds 4–5	80.08%	`phase2_shifts/diagnosis...`	Missed 85% target due to Nominal/Action confusion
Diagnosis Classifier	Overall Accuracy	Seed 6 (Held-Out)	89.05%	`phase2_shifts/...seed6...`	Confirmatory test on pristine seed; passes 85% target
Observation Filter	Recovery Ratio	Seed 5 (In-Sample)	+15.96%	`phase2_shifts/mitigation...`	Wait dropped from 295.63 to 255.81 s/step
Transition Stretcher	Recovery Ratio	Seed 5 (In-Sample)	+21.99%	`phase2_shifts/mitigation...`	Wait dropped from 51.57 to 50.38 s/step
Action Lookahead	Recovery Ratio	Seed 5 (In-Sample)	-46.62%	`phase2_shifts/mitigation...`	Harmful; lookahead caused chattering; safely gated off
Fallback Governor	Recovery Ratio	Seed 4 (Synthetic)	+99.79%	`phase2_shifts/mitigation...`	Inverted Q-value test; mitigated wait is ~61x nominal

9. Project Completion Status & Roadmap to Final Review

Quantitative Completion Formula:
Project completion is evaluated objectively using weighted milestone completion: $C = w_1 P_1 + w_2 P_2 + w_3 P_3$, where $w_1 = 0.30$ (Phase 1), $w_2 = 0.35$ (Phase 2), and $w_3 = 0.35$ (Phase 3).
$$C = (0.30 \times 1.00) + (0.35 \times 0.85) + (0.35 \times 0.25) = 0.300 + 0.2975 + 0.0875 = \mathbf{68.5\%}$$

Phase / Work Item	Completion	Primary Owner	Delivered Assets & Status	Remaining Tasks for Final Review
Phase 1: Baselines & Reprod.	100%	**Ayyappa Reddy**	5-seed DQN checkpoints, baselines, left-turn fix, fairness sweep	Complete. Ready for final archival.
Phase 2: Shift Resilience	85%	**Sriram**	4-way shift engine, 3-sigma monitor, RF classifier, targeted mitigations	Add rolling queue variance to eliminate nominal/action confusion; expand mid-episode sensitivity.
Phase 3: Closed-Loop System	25%	**Vivek Reddy**	Integrated controller, fallback governor, pilot runs on Seeds 7–9	Execute full multi-seed benchmark across pristine seeds 16–35; complete component ablations.

10. Glossary of Terms & Mathematical Notation

• **DQN (Deep Q-Network):** A model-free reinforcement learning algorithm that approximates the optimal action-value function Q(s, a) using a deep neural network, trained via experience replay and temporal difference loss.

• **SUMO (Simulation of Urban MObility):** An open-source, highly portable, microscopic road traffic simulation package designed to handle large networks and individual vehicle kinematics.

• **TraCI (Traffic Control Interface):** The TCP-based client-server protocol providing live programmatic control and state inspection of SUMO simulations from Python.

• **Max-Pressure Control:** A classical state-feedback traffic signal control policy that selects phases to maximize differential queue pressure between incoming and outgoing lanes, provably stabilizing queues under infinite buffer assumptions.

• **min_green:** The minimum number of decision steps a selected green phase must be held before switching. In this project, 1 decision step = 10 simulation seconds of green.

• **Phase 1 Metric:** The sum of instantaneous vehicle queues across all incoming approaches at each decision step, accumulated and averaged across the episode (units: queue-sum seconds per step).

• **Tripinfo Metric:** The exact total waiting time spent by a vehicle with speed below 0.1 m/s across its entire trip, averaged per completed vehicle (units: seconds per vehicle).

• **S/T/A/R Taxonomy:** The four-way distribution shift decomposition across State, Transition, Action, and Reward dimensions (conceptual taxonomy credited to Sim2Signal: Al Rafi et al., 2026).

• **Recovery Ratio (rho):** The normalized recovery metric defined as (W_shifted - W_mitigated) / (W_shifted - W_nominal) * 100%, measuring the percentage of shift-induced delay recovered by mitigation.

• **Fallback Governor:** A supervisory fail-safe controller that overrides the neural network policy during severe anomalies or policy collapse, executing tuned Max-Pressure (min_green=1).